

README

HelixCode API Reference — Zero-Bluff Phase 5

Audience: Developers integrating against HelixCode internals. **Scope:** 20 key internal packages from the inner Go module (`helix_code/helix_code/`). **Generation:** Content sourced from `go doc ./internal/<pkg>` — real signatures, no invention (CONST-035). **Companion:** REST API at [docs/COMPLETE_API_REFERENCE.md](#) (1,362 lines). **Last updated:** 2026-05-12

Module Identity

```
module dev.helix.code
go 1.26
```

Located at `helix_code/helix_code/` (inner submodule). The meta-repo root has a thin module also called `dev.helix.code` on Go 1.25.2 used only for governance tooling.

Package Index

Package	Purpose	Doc page
<code>internal/approval</code>	Tool-approval gate, 4 modes × 4 levels	§1
<code>internal/autocommit</code>	Aider-style per-edit git auto-commit	§2
<code>internal/clarification</code>	LLM-driven ambiguity detection (CONST-046)	§3
<code>internal/editor</code>	Smart file editing — 4 formats, model-aware	§4
<code>internal/task</code>	Distributed task management + checkpointing	§5
<code>internal/projectmemory</code>	Codex-style project memory loader + hot-reload	§6
<code>internal/plantree</code>	Plandex-style branching plan trees	§7
<code>internal/kilocode</code>	AST-aware multi-file refactoring	§8
<code>internal/roocode</code>	RooCode port — delegation, generation, review	§9
<code>internal/plugins</code>	Plugin loader + registry	§10

Package	Purpose	Doc page
internal/repomap	Semantic codebase mapping (tree-sitter)	§11
internal/quality	Quality gates + scoring	§12
internal/voice	Speech-to-text voice input	§13
internal/llm	LLM provider interface + adapters	§14
internal/providers	15 LLM provider implementations	§15
internal/verifier	LLMsVerifier integration (CONST-036–040)	§16
internal/telemetry	OpenTelemetry SDK wrapper	§17
internal/session	Session transcript + resume	§18
internal/tools	Tool interface + registry	§19
internal/commands	Slash command registry	§20

To regenerate any section locally:

```
cd HelixCode && go doc ./internal/<pkg>
```

1. internal/approval

Tool-approval gate. The gate is a runtime check applied to every tool invocation. Two inputs determine the outcome: the tool’s required action level (read-only, edit, run, all) and the user’s currently active approval mode (suggest, auto-edit, full-auto, dangerously-bypass). The gate produces a `Decision` (allow / deny / prompt) the executor enforces.

Key exports:

- `const EnvVarName = "HELIXCODE_APPROVAL"`
- `var ErrInvalidMode, ErrNoPromptResponder, ErrSandboxRequired`
- `func Select(input SelectorInput) (ApprovalMode, ResolvedSource, error)` — flag > env > config > default precedence
- `type ApprovalMode string` — constants `ModeSuggest`, `ModeAutoEdit`, `ModeFullAuto`, `ModeDangerouslyBypass`
- `type ApprovalLevel int` — `LevelReadOnly`, `LevelEdit`, `LevelRun`, `LevelAll`
- `type ApprovalManager struct{...}` — `NewApprovalManager(opts) (*ApprovalManager, error)`
- `type Decision int` — `DecisionAllow`, `DecisionDeny`, `DecisionPrompt`
- `type PromptResponder interface{...}` — user prompt adapter for auto-edit mode

Anchor: CONST-035. All 4 modes mirror codex semantics verbatim; no mode is a marketing label.

2. internal/autocommit

Aider-style per-edit git auto-commit (F22). The registry calls `MaybeCommit(ctx, CommitContext)` after every successful edit-class tool invocation.

Pipeline:

1. Check `SkipRequested` flag (and `SkipParamKey` in args).
2. Check atomic-bool enabled flag (env at startup; mutated by `/git_auto_commit on/off` at runtime).
3. Verify working dir is a git repo.
4. `git status --porcelain` — clean tree → no-op.
5. Stage `MutatedPaths` (or all dirty paths if list empty).
6. `git diff --staged` → `MessageSummariser.Summarise` → 1-line subject (≤ 72 chars).
7. Run `SecretFilter` over the subject.
8. Build full message: `subject + "\n\n" + CoAuthorTrailer`.
9. `git commit -m <message>` and read back new SHA.
10. Return `CommitResult{SHA, Subject, Files, Skipped:false}`.

`SecretFilter` patterns: AWS access keys (`AKIA[0-9A-Z]{16}`), OpenAI / generic (`sk-[A-Za-z0-9]{20,}`), Slack tokens (`xox[baprs]-[A-Za-z0-9-]{10,}`), GitHub PATs (`gh[pousr]_[A-Za-z0-9]{36}`).

Anchors: CONST-042 (no secret leak), CONST-043 (no force push — diff body never leaves host).

3. internal/clarification

LLM-driven ambiguity detection (F19, CONST-046). All questions are LLM-generated to be locale-aware; no hardcoded English strings.

Exports:

- `type Engine struct{...}` — `NewEngine(llmProvider *litellm.UnifiedProvider) *Engine`
- `type Question struct{...}` — `Type QuestionType`
- `type QuestionType string` — `YesNo, OpenEnded, MultipleChoice, FreeForm`
- `type Answer struct{...}`
- `type Session struct{...}` — tracks Q&A within a single clarification flow
- `type QuestionGenerator struct{...}` — `NewQuestionGenerator(llmProvider)`

4. internal/editor

Smart file editing (F17). Four formats, model-aware selection:

- `EditFormatDiff` — Unix unified diff
- `EditFormatWhole` — whole-file replacement
- `EditFormatSearchReplace` — pattern-based, optional regex
- `EditFormatLines` — line-range edits

Format selection helpers:

```
format := editor.SelectFormatForModel("gpt-4o")           //
        EditFormatDiff
format = editor.SelectFormatForModel("claude-3-sonnet")    //
        EditFormatSearchReplace
ok := editor.SupportsFormat("gpt-4", editor.EditFormatDiff)
cap := editor.GetModelCapability("claude-3-opus")
```

Applying edits:

```
ed, _ := editor.NewCodeEditor(editor.EditFormatDiff)
ed.ApplyEdit(editor.Edit{FilePath: "test.go", Format:
    editor.EditFormatDiff, Content: "..."})
```

Built-in validation, backup support, and syntax checks. See `helix_code/internal/editor/README.md`.

5. internal/task

Distributed task management with checkpointing, dependencies, priority queueing, and Redis caching.

```
db := database.Connect(cfg); redis := redis.Connect(cfg)
manager := task.NewTaskManager(db, redis)
t, err := manager.CreateTask(
    task.TaskTypeBuilding,
    map[string]any{"file": "main.go"},
    task.PriorityHigh,
    task.CriticalityNormal,
    []uuid.UUID{}, // deps
)
```

Types: `TaskTypePlanning`, `TaskTypeBuilding`, `TaskTypeTesting`, `TaskTypeRefactoring`, `TaskTypeDebugging`, `TaskTypeDesign`, `TaskTypeDiagram`, `TaskTypeDeployment`, `TaskTypePorting`.

Priorities: `PriorityLow` (1), `PriorityNormal` (5), `PriorityHigh` (10), `PriorityCritical` (20).

Statuses: `Pending`, `Assigned`, `Running`, `Completed`, `Failed`, `Paused`, `WaitingForWorker`, `WaitingForDeps`.

`TaskQueue` provides priority-based scheduling; `CheckpointManager` enables failure recovery.

6. internal/projectmemory

Codex-style project memory (F24). Loads a project-root Markdown file (`helixcode.md` / `codex.md` / `AGENTS.md`, first-found wins) plus a per-user overlay at `$XDG_CONFIG_HOME/helixcode/memory.md` into every LLM call's system prompt.

Components:

- `Memory` — immutable value type with content, paths, truncation flags, load timestamp.
- `MemoryLoader` — parent-walk discovery (`cwd` → `parent` → `git root`) + user overlay reader. Missing files are NOT errors.
- `MemoryRegistry` — `atomic.Pointer[Memory]`; lock-free `Snapshot`, `mu-serialised Reload`. Implements `MemorySnapshotter`.
- `MemoryWatcher` — `fsnotify` wrapper with 200 ms debounce; watches parent directories so atomic-write editors (`vim`, `emacs`) survive renames. Graceful degrade when `fsnotify` unavailable.

Constants: `DebounceWindow = 200ms`, `MaxMemoryBytes = 64 * 1024`.

Anchor: `CONST-035` (runtime evidence) + `CONST-042` (memory contents never logged at INFO).

7. internal/plantree

Plandex-style branching plan trees (F25). Plans are trees of `PlanNode` (title, description, status, children) serialised to `.helixcode/plans/<name>.json`.

Six `tools.Tool` implementations: `PlanCreateTool`, `PlanShowTool`, `PlanListTool`, `PlanBranchTool`, `PlanMergeTool`, `PlanDeleteTool`.

Helpers:

```
tree, _ := plantree.CreateTree("feature-x", "Title",
    "Description")
plantree.BranchNode(tree, parentID, "subtask", "details")
plantree.MergeNode(tree, childID)
plantree.VerifyTree(tree)           // → VerifyResult{Issues:
    []PlanIssue}
plantree.CompactTree(tree, summariser) // reuses F01
    AutoCompactor
plantree.RenderTree(node, depth)
```

Limits: `MaxNodes = 500`.

8. internal/kilocode

AST-aware multi-file refactoring (F28). Cross-file rename via tree-sitter queries + atomic F17 smart-edits; impact analysis with call graph + blast radius.

Exports:

- `BuildCallGraph(rootDir) (*CallGraph, error)` — full repo call graph
- `NewImpactAnalyzer(rootDir) (*ImpactAnalyzer, error)` — impact + blast radius
- `NewRenameEngine(rootDir) *RenameEngine` — cross-file rename
- `NewRefactorer(rootDir) *Refactorer` — extract / inline / move

Tools: `KiloRenameTool`, `KiloImpactTool`, `KiloMultiEditTool`.

Limits: `MaxRenameFiles = 500`.

9. internal/roocode

RooCode port (F29). Task delegation via F15 subagents, template-based code generation with LLM content, diff-based code review, conversation-aware memory.

Exports: `TaskDelegator`, `CodeGenerator`, `CodeReviewer`, `ConversationStore`, plus tools `RooDelegateTool`, `RooGenerateTool`, `RooBootstrapTool`.

10. internal/plugins

Plugin loader + registry. Plugins declared by `manifest.yaml` with `name` / `entry` / `permissions`; loaded from `~/.config/helixcode/plugins/`.

```
ldr := plugins.NewLoader("~/config/helixcode/plugins/")
reg := plugins.NewRegistry()
plugins.ExecutePlugin(ctx, plg, "run", []string{"--flag"})
```

11. internal/repomap

Semantic codebase mapping for AI context (F27). Tree-sitter based.

```
cfg := repomap.DefaultConfig()
cfg.TokenBudget = 8000; cfg.MaxFiles = 100
rm, _ := repomap.NewRepoMap("/path", cfg)
ctxs, _ := rm.GetOptimalContext("implement user auth",
    changedFiles)
```

Supported languages: Go, Python, JavaScript/TypeScript, Java, C/C++, Rust, Ruby. Symbols: functions, methods, classes, structs, interfaces, enums, traits, modules, constants.

`FileRanker` scores by name match, doc content, recency, import relationships. `RepoCache` provides TTL-bounded caching with async disk persistence; callers MUST invoke `(*RepoCache).Close()` (or `Wait()`) before disposing the cache directory to drain in-flight writes — otherwise fire-and-forget save goroutines may still be writing into a doomed directory at process/test exit.

12. internal/quality

Quality gates + scoring.

- type `QualityGate struct{...}` — `DefaultGate()`, `StrictGate()`
- type `Scorer struct{}` — `NewScorer()`
- type `ScoreResult struct{...}`
- type `History struct{...}` — `NewHistory(path)` persists scores per session

13. internal/voice

Speech-to-text voice input (F27). Audio capture via arecord / sox, transcription via OpenAI Whisper API with whisper.cpp local fallback.

Exports: VoiceRecorder, VoiceTranscriber, VoiceConfig, TranscriptionResult. Tools: VoiceStartTool, VoiceStopTool, VoiceTranscribeTool.

Engines: EngineWhisperAPI, EngineWhisperCPP.

Default model: whisper-1.

14. internal/llm

LLM provider interface used by every adapter.

```
type Provider interface {
    Generate(ctx context.Context, req *GenerateRequest)
        (*GenerateResponse, error)
    GenerateStream(ctx context.Context, req *GenerateRequest)
        (<-chan *Chunk, error)
    GetModels() ([]Model, error)
    HealthCheck(ctx context.Context) error
}
```

15. internal/providers

15 provider implementations: anthropic, openai, gemini, bedrock, azure_openai, vertexai, groq, mistral, deepseek, xai, openrouter, ollama, llamacpp, litellm, characterai. Each implements llm.Provider.

CONST-036-040: provider catalogue + model status MUST come from LLMsVerifier; hardcoded model lists are forbidden.

16. internal/verifier

LLMsVerifier client. Provides the single source of truth for model + provider metadata and capability flags (MCP, LSP, ACP, Embedding, RAG, Skills, Plugins).

17. internal/telemetry

OpenTelemetry SDK wrapper. Spans wrap every tool invocation; metrics expose tokens-in, tokens-out, request latency, sandbox denials, approval-gate denials.

`HELIXCODE_OTEL_ENDPOINT=http://localhost:4317 ./bin/cli`

18. internal/session

Session transcript + resume (F11). Transcripts stored at `~/.local/share/helixcode/sessions/<id>/.cli` sessions list and `cli sessions resume <id>` for user-facing access.

19. internal/tools

Tool registry. The contract every tool implements:

```
type Tool interface {
    Name() string
    Description() string
    Schema() *jsonschema.Schema
    RequiresApproval() approval.ApprovalLevel
    Validate(args map[string]any) error
    Execute(ctx context.Context, args map[string]any) (*Result,
        error)
}
```

MustRegister, Get, List, Filter, Execute are the registry surface.

20. internal/commands

Slash command registry. Loads built-in commands plus user-defined ones from `~/.config/helixcode/commands/*.md`. See [docs/developer_guide/README.md](https://go.dev/doc/devel/release) §4.

Sources verified

Sources verified 2026-05-29: <https://go.dev/doc/devel/release> (Go 1.26.0 GA 2026-02-10, latest patch go1.26.3 — confirms the documented go 1.26 inner module and Go 1.25.2 thin root module match CLAUDE.md §3.1; no stale pin to fix). Negative finding: this API reference documents internal package signatures only — no external service endpoints, third-party API URLs, or downloadable-dependency versions are present to cross-reference; the version authority is the project's own go.mod files and CLAUDE.md §3.1.